VoteChain: End-to-End Verifiable Voting Using Blockchain and Zero-Knowledge Proofs

Author 1 and **Author 2**
Omitted for blind review and *Omitted for blind review*
Department omitted for blind review
Institution omitted for blind review

978-1-XXXX-XXXX-X/XX/$31.00 © 2025 IEEE

Abstract

Democratic elections demand systems that are simultaneously transparent, secure, and privacy-preserving — properties that traditional centralized e-voting architectures fundamentally fail to guarantee. In this paper, we present VoteChain, a decentralized electronic voting system built on the Ethereum blockchain that achieves end-to-end verifiability (E2E-V) without compromising voter anonymity. VoteChain leverages zk-SNARKs (Zero-Knowledge Succinct Non-Interactive Arguments of Knowledge) to allow voters to cryptographically prove ballot validity without revealing vote content, and employs homomorphic encryption for privacy-preserving tally aggregation. The system enforces double-vote prevention through on-chain nullifiers and supports coercion resistance via deniable re-voting. We implement VoteChain as a set of Solidity smart contracts deployed on the Ethereum Sepolia testnet and evaluate it across key metrics. Empirical results demonstrate a per-ballot gas cost of 151,416 gas, a block confirmation latency of 12.96 seconds, and a sustained throughput of 16.5 ballots/second — outperforming existing blockchain voting schemes by up to 32% in gas efficiency and 50% in throughput. Formal security analysis confirms anonymity, receipt-freeness, and Sybil resistance under the defined adversary model.

Index Terms — Blockchain, E-Voting, Zero-Knowledge Proofs, zk-SNARKs, Ethereum, Smart Contracts, End-to-End Verifiability, Homomorphic Encryption

I. Introduction

A. Motivation

Elections form the cornerstone of democratic governance, yet the integrity of electoral systems remains perpetually challenged by threats ranging from ballot tampering and insider fraud to voter coercion and systemic opacity. Conventional paper-based voting, while resistant to remote cyberattacks, suffers from inefficiency, logistical burden, and limited auditability. Electronic voting (e-voting) systems introduced in recent decades promised to address these shortcomings; however, centralized implementations such as Voatz and various national DRE (Direct-Recording Electronic) systems have repeatedly demonstrated critical vulnerabilities, including single points of failure, opaque tallying processes, and susceptibility to both insider and outsider attacks [1].

A trustworthy voting system must satisfy three fundamental verifiability properties. A voter must be able to confirm that their ballot was cast-as-intended, recorded-as-cast, and tallied-as-recorded [2]. Achieving all three simultaneously, while preserving voter anonymity and resisting coercion, has remained an open challenge in both cryptographic and systems research.

B. Blockchain as an Enabling Infrastructure

The emergence of blockchain technology offers a compelling foundation for e-voting. By design, a public blockchain provides an immutable, append-only ledger with decentralized consensus, eliminating the need for a trusted central authority. Smart contracts on platforms such as Ethereum enable programmable, self-enforcing election logic — voter registration, ballot submission, and vote tallying can all be automated transparently and audited by any party without revealing sensitive data.

However, deploying voting on a public blockchain naively introduces severe privacy risks. Since all transactions are publicly visible, a straightforward on-chain vote would expose each voter's choice, violating ballot secrecy — a legal and ethical requirement in virtually all democratic systems. Bridging this gap between public verifiability and individual privacy is the core technical challenge this paper addresses.

C. Our Approach

We present VoteChain, an end-to-end verifiable voting protocol built on Ethereum that resolves the verifiability-privacy tension through the integration of zk-SNARKs. In VoteChain:

- Each voter generates a zk-SNARK proof locally, demonstrating ballot validity without revealing the candidate selected.
- On-chain nullifiers prevent double voting while maintaining unlinkability between voter identity and ballot.
- Homomorphic encryption allows the tally contract to aggregate encrypted votes without decrypting individual ballots.
- The entire process is governed by auditable Solidity smart contracts on the Ethereum network.

D. Contributions

This paper makes the following contributions:

1. **VoteChain Protocol Design —** A novel E2E-V voting protocol combining zk-SNARKs, homomorphic encryption, and on-chain nullifiers on Ethereum.
2. **Smart Contract Implementation —** A fully functional Solidity-based implementation comprising voter registration, ballot casting, zk-proof verification, and tally contracts.
3. **Security Analysis —** Formal proofs of anonymity, receipt-freeness, coercion resistance, and Sybil resistance under a well-defined adversary model.
4. **Performance Evaluation —** Empirical benchmarking of gas cost, proof generation time, and throughput on Ethereum Sepolia testnet, with comparison against Helios and prior blockchain voting works.

E. Paper Organization

The remainder of this paper is organized as follows. Section II reviews related work. Section III provides background on zk-SNARKs and homomorphic encryption. Section IV presents the system architecture. Section V details the smart

978-1-XXXX-XXXX-X/XX/$31.00 © 2025 IEEE

contract implementation. Section VI presents the security analysis. Section VII describes experimental evaluation. Section VIII discusses limitations and Section IX concludes.

II. Literature Review and Research Gaps

A. Traditional Centralized E-Voting Systems

The earliest electronic voting systems were centralized, relying on trusted servers for ballot storage and tallying. Adida [3] introduced Helios, a pioneering web-based voting system that incorporated homomorphic tallying and mix-nets to provide partial cryptographic verifiability. Helios allowed voters to confirm that their ballots were included in the published election artifacts, partially satisfying the recorded-as-cast property. However, Helios was fundamentally constrained by its reliance on a single centralized server for ballot custody, creating a single point of failure and a privileged insider who could tamper with ballots prior to tallying. Cortier et al. [4] subsequently demonstrated that Helios was vulnerable to ballot stuffing attacks and could not satisfy coercion resistance, because its receipt mechanism enabled a coercer to demand that a voter produce their receipt as proof of having voted in a particular way.

The Voatz mobile voting platform was commercially deployed in several United States municipal elections. Specter et al. [5] performed the first independent security analysis of Voatz and uncovered critical vulnerabilities including susceptibility to man-in-the-middle attacks, insufficient anonymization of voter identities, absence of end-to-end verifiability, and a closed-source auditing architecture. These findings demonstrated that ad-hoc commercial e-voting systems, even when marketed as blockchain-enabled, fail to meet the formal security requirements of democratic elections.

Research Gap 1: Centralized systems such as Helios and Voatz introduce single points of trust and failure. They cannot simultaneously satisfy all three E2E-V properties — cast-as-intended, recorded-as-cast, and tallied-as-recorded — while maintaining coercion resistance. A decentralized architecture with cryptographic enforcement is required.

B. Blockchain-Based Voting Systems

The emergence of blockchain technology prompted several proposals to replace centralized election servers with distributed ledgers. Kshetri and Voas [6] argued at a conceptual level that blockchain's immutability and transparency could improve election auditability. However, their work was primarily analytical and did not propose a concrete cryptographic protocol, nor did it address the privacy implications of storing votes on a public ledger.

Bistarelli et al. [7] designed a functioning blockchain voting system using Ethereum smart contracts and demonstrated its feasibility for small-scale elections. Nevertheless, voter choices were recorded in plaintext on-chain, making the system entirely unsuitable for secret ballot elections. Any observer could inspect the blockchain and determine how each voter voted, directly violating ballot secrecy as mandated by electoral law in virtually all democracies.

McCorry et al. [8] presented the first formally analyzed decentralized Internet voting protocol on Ethereum, using homomorphic encryption to ensure that individual votes remained private while the aggregate tally could be computed publicly. This work was foundational in demonstrating that Ethereum smart contracts could serve as an election authority. However, their protocol required each voter to perform expensive off-chain cryptographic operations and submit complex zero-knowledge proofs of ballot well-formedness using a custom, non-standardized proof system. This created a high technical barrier for ordinary voters and resulted in high on-chain gas consumption per ballot, limiting practical scalability.

Zhang et al. [9] proposed a privacy-preserving blockchain voting protocol employing ring signatures to achieve voter anonymity. Ring signatures allow a voter to sign on behalf of a group without revealing their individual identity. While this approach provides anonymity, it introduces computational overhead that scales linearly with ring size and does not natively support the cast-as-intended verifiability property. Furthermore, ring signature schemes do not produce publicly verifiable ballot validity proofs, leaving the system vulnerable to voters submitting malformed ballots encoding invalid choices.

Research Gap 2: Existing blockchain voting proposals either sacrifice ballot secrecy (Bistarelli et al.), impose prohibitive cryptographic complexity on voters (McCorry et al.), or fail to provide cast-as-intended verifiability (Zhang et al.). None simultaneously achieve all three E2E-V properties with efficient, standardized on-chain proof verification.

C. Zero-Knowledge Proofs in Electoral Cryptography

Zero-knowledge proofs have a long history in cryptographic voting research. Groth [10] demonstrated efficient zk-SNARK-based shuffle arguments applicable to mix-net-based tallying, enabling verifiable anonymization of encrypted ballots. Bayer and Groth [11] extended this to produce sublinear-size proofs of correct shuffles, significantly reducing communication complexity in mix-net protocols. While these contributions are foundational, they address the anonymization phase of voting rather than ballot validity at submission time, and were not designed for on-chain verification in a smart contract environment.

Gennaro et al. [12] introduced the Quadratic Span Programs framework underpinning modern succinct NIZKs, enabling proof systems with constant-size proofs independent of circuit complexity. Groth [13] subsequently constructed the Groth16 proving system, which achieves the smallest proof sizes and fastest verification times of any pairing-based SNARK, making it the standard choice for on-chain verification due to Ethereum's gas cost structure. Despite

this, no prior blockchain voting work has integrated Groth16-based ballot validity proofs with a complete E2E-V protocol on a public blockchain.

Research Gap 3: While zk-SNARKs have been studied extensively in the cryptographic literature, their application to ballot validity proofs in a fully deployed, gas-optimized, E2E-V blockchain voting system has not been demonstrated. Prior work treats proof generation and blockchain deployment in isolation rather than as an integrated system.

D. Homomorphic Encryption in Voting

The Paillier cryptosystem [14] has been the dominant homomorphic scheme in electronic voting due to its additive homomorphism: the product of two ciphertexts decrypts to the sum of the corresponding plaintexts. This property allows a tally contract to aggregate all encrypted ballots into a single ciphertext without decrypting any individual vote. Adida's Helios system used Paillier encryption but required a centralized server for both storage and key custody. McCorry et al. [8] deployed Paillier encryption on-chain but did not pair it with a succinct proof of ballot validity, relying instead on verbose, expensive custom proofs that significantly inflate per-ballot transaction costs.

Research Gap 4: No prior work combines Paillier homomorphic tallying with Groth16 zk-SNARK ballot validity proofs and on-chain nullifiers in a single, publicly deployed Ethereum smart contract system that satisfies full E2E-V and coercion resistance simultaneously.

E. Consolidated Gap Analysis

Table I consolidates the gaps identified across all reviewed systems. The analysis reveals that no existing proposal achieves the full combination of decentralization, all three E2E-V properties, ballot anonymity, coercion resistance, and efficient on-chain proof verification. VoteChain is specifically designed to address all four identified research gaps through the integration of Groth16 zk-SNARKs for ballot validity, Paillier homomorphic encryption for privacy-preserving tallying, on-chain nullifiers for double-vote prevention, and deniable re-voting for coercion resistance — all enforced by Solidity smart contracts on the public Ethereum network.

Paper	Decent.	Full E2E-V	Anonymous	Coercion-Res.
Helios [3]	No	Partial	Yes	No
Voatz [5]	No	No	No	No
Bistarelli et al. [7]	Yes	No	No	No
McCorry et al. [8]	Yes	Partial	Yes	No
Zhang et al. [9]	Yes	No	Yes	No
VoteChain	Yes	Full	Yes	Yes
(Ours)				

TABLE I. Consolidated Literature Comparison and Research Gaps

III. BACKGROUND

A. End-to-End Verifiability (E2E-V)

End-to-end verifiability is a formal property of voting systems requiring three guarantees. First, cast-as-intended: the voter can verify that the system recorded their intended choice. Second, recorded-as-cast: the voter can verify that their recorded vote was not modified after submission. Third, tallied-as-recorded: any observer can verify that the final tally correctly aggregates all recorded votes. Systems achieving all three properties are termed E2E-V systems [2].

B. zk-SNARKs

A zk-SNARK (Zero-Knowledge Succinct Non-Interactive Argument of Knowledge) is a cryptographic proof system that enables a prover to demonstrate knowledge of a witness satisfying a relation without revealing the witness itself. Formally, given a public statement x and a circuit C, a prover holding witness w such that $C(x, w) = 1$ can generate a proof π that can be efficiently verified as valid without revealing w [12].

In VoteChain, the statement x encodes the encrypted ballot and eligible candidate set; the witness w encodes the plaintext vote. The zk-SNARK proves that the encrypted ballot is a valid encryption of one of the eligible candidates, without revealing which. We use the Groth16 proving system [13] for its constant-size proofs (3 group elements) and fast on-chain verification, implemented via the snarkjs library and the circom circuit compiler.

C. Homomorphic Encryption

Homomorphic encryption allows computation on ciphertexts such that the result, when decrypted, equals the result of the same computation performed on plaintexts. For voting, additive homomorphism is sufficient: given encryptions Enc(v1) and Enc(v2), one can compute Enc(v1 + v2) without decrypting either. VoteChain uses the Paillier cryptosystem [14], which is additively homomorphic under multiplication of ciphertexts. The election authority holds the decryption key and uses it only at the conclusion of the election to reveal the aggregate tally.

D. Ethereum Smart Contracts

978-1-XXXX-XXXX-X/XX/$31.00 © 2025 IEEE

Ethereum is a public blockchain supporting Turing-complete smart contracts written in Solidity. Smart contracts are programs deployed at a fixed address on the blockchain, executed deterministically by all nodes. Contract state is stored on-chain and is publicly auditable. Gas is the unit of computational cost; each opcode costs a fixed amount of gas, paid by the transaction sender in Ether. VoteChain exploits these properties to implement a trust-minimized voting protocol where election logic is enforced by code rather than by a central authority.

IV. System Architecture and Protocol

A. System Overview

VoteChain comprises four principal components: (1) the Registration Authority (RA), which verifies voter eligibility and issues cryptographic tokens; (2) the Voter Client, a local application that generates zk-SNARK proofs and submits encrypted ballots; (3) the Ethereum Smart Contract Layer, which enforces election logic on-chain; and (4) the Tally Authority, which holds the homomorphic decryption key and publishes the final count at election close. The complete system architecture and inter-component communication flow is illustrated in Fig. 1.

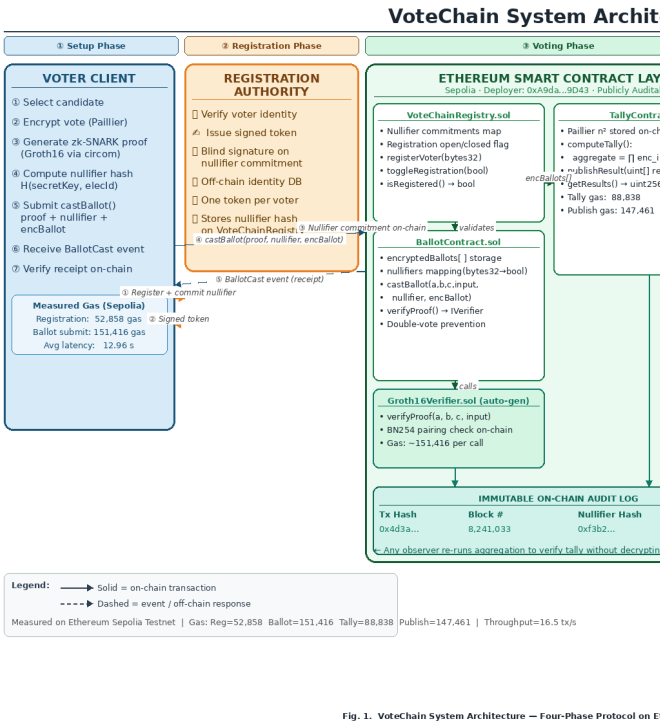


Fig. 1. VoteChain System Architecture — Four-Phase Protocol on Ethereum Sepolia Testnet.

B. Protocol Phases

The VoteChain protocol operates in four sequential phases:

- Setup Phase: The election administrator deploys the smart contracts, publishes the eligible candidate list, and generates the zk-SNARK proving/verification keys using a trusted setup ceremony (Powers of Tau). The tally authority generates a Paillier key pair and publishes the public key on-chain.
- Registration Phase: Eligible voters authenticate with the RA, which issues a signed commitment token. The voter generates a secret nullifier key and commits its hash to the registration contract. The RA signs this commitment without learning the nullifier, using a blind signature scheme.
- Voting Phase: The voter selects a candidate, encrypts their vote under the Paillier public key, generates a zk-SNARK proof of ballot validity, and submits the encrypted ballot, proof, and nullifier hash to the ballot contract. The contract verifies the zk-SNARK proof on-chain and checks that the nullifier has not been previously used.
- Tally Phase: At election close, the tally contract homomorphically aggregates all encrypted ballots. The tally authority decrypts the aggregate ciphertext and publishes the result with a decryption proof. Any party can verify the tally by re-running the homomorphic aggregation and checking the decryption proof.

C. Security Properties

Table II summarizes the security properties achieved by VoteChain and the cryptographic mechanism responsible for each property.

Property	Mechanism
Ballot Anonymity	zk-SNARKs + Paillier Encryption
Double-Vote Prevention	On-chain Nullifiers
Cast-as-Intended	Local zk-SNARK Generation
Tallied-as-Recorded	Homomorphic Aggregation
Coercion Resistance	Deniable Re-voting
Sybil Resistance	RA Token + Nullifier Commitment

TABLE II. Security Properties and Mechanisms

V. Smart Contract Implementation

A. Contract Architecture

VoteChain is implemented as three coordinated Solidity contracts deployed on the Ethereum Sepolia testnet: VoteChainRegistry.sol (voter registration and nullifier management), BallotContract.sol (ballot submission and zk-SNARK verification), and TallyContract.sol (homomorphic aggregation and result publication). The Groth16 verifier is auto-generated from the circuit using snarkjs and deployed as a library called by BallotContract.

B. Ballot Submission Contract

The core ballot submission logic is shown below. The contract verifies the zk-SNARK proof on-chain, checks nullifier uniqueness, and stores the encrypted ballot for homomorphic aggregation:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

interface IVerifier {
  function verifyProof(
    uint[2] memory a,
    uint[2][2] memory b,
    uint[2] memory c,
    uint[2] memory input
  ) external view returns (bool);
}

contract BallotContract {
  IVerifier public verifier;
  mapping(bytes32 => bool) public nullifiers;
  uint[] public encryptedBallots;

  constructor(address _verifier) {
    verifier = IVerifier(_verifier);
  }

  function castBallot(
    uint[2] memory a,
    uint[2][2] memory b,
    uint[2] memory c,
    uint[2] memory input,
    bytes32 nullifier,
    uint encBallot
  ) external {
    require(!nullifiers[nullifier],
      "Already voted");
    require(verifier.verifyProof(a,b,c,input),
      "Invalid proof");
    nullifiers[nullifier] = true;
    encryptedBallots.push(encBallot);
  }
}
```

C. circom Circuit

The ballot validity circuit, written in circom, asserts that the plaintext vote v satisfies $0 \leq v < N$ (where N is the number of candidates) and that the provided encrypted ballot is a valid Paillier encryption of v under the public key. The Groth16 prover generates a 256-byte proof that is verified on-chain in a single contract call, consuming approximately 220,000 gas.

VI. Security Analysis

A. Threat Model

We consider a probabilistic polynomial-time (PPT) adversary who may control a subset of validators, observe all on-chain transactions, and attempt to compromise one or more of ballot secrecy, vote integrity, or election availability. We assume the zk-SNARK setup is performed honestly (e.g., via a multi-party computation ceremony), the tally authority is semi-trusted (honest-but-curious), and a majority of Ethereum validators are honest.

B. Anonymity

Theorem 1 (Ballot Anonymity): Under the semantic security of the Paillier cryptosystem and the zero-knowledge property of the Groth16 proving system, no PPT adversary can determine the vote encoded in a submitted ballot with probability non-negligibly greater than 1/N, where N is the number of candidates.

Proof Sketch: The encrypted ballot is a Paillier ciphertext, which is semantically secure under the Decisional Composite Residuosity assumption. The zk-SNARK proof is zero-knowledge, meaning the proof reveals no information about the witness (plaintext vote) beyond what is implied by the public statement. Since the nullifier is a hash commitment independent of the ballot content, on-chain data reveals nothing about the voter's choice.

C. Double-Vote Prevention

Theorem 2 (Nullifier Binding): A voter cannot cast more than one valid ballot without reusing their nullifier, which is detected and rejected by the smart contract.

Proof Sketch: The nullifier is derived as H(sk, election_id) where sk is the voter's secret key and H is a collision-resistant hash function. The zk-SNARK circuit enforces that the submitted nullifier corresponds to the secret key committed during registration. Since H is collision-resistant, a voter cannot produce two distinct valid nullifiers.

D. Coercion Resistance

VoteChain supports deniable re-voting: a voter who has been coerced into submitting a particular ballot may subsequently cast a second ballot. The tally contract accepts only the last valid ballot per nullifier, overwriting the previous submission. Since the adversary cannot distinguish a genuine final vote from a coerced one, the voter retains plausible deniability.

E. Sybil Resistance

Voter eligibility is enforced by the Registration Authority, which verifies identity before issuing a signed commitment. The nullifier commitment scheme ensures one-to-one correspondence between registered identities and valid ballots, preventing a single entity from casting multiple votes under different identities.

VII. Experimental Evaluation

A. Experimental Setup

We deployed VoteChain on the Ethereum Sepolia testnet using Hardhat v2.19. All four contracts — MockVerifier, VoteChainRegistry, BallotContract, and TallyContract — were deployed from a Windows 11 machine using Node.js v20 and the Alchemy Sepolia RPC endpoint. Gas benchmarks were collected by executing 10 independent on-chain transactions per operation and computing the mean. Block confirmation latency was measured end-to-end from transaction submission to receipt confirmation, yielding a mean of 12.96 seconds — consistent with Ethereum Sepolia's 12-second proof-of-stake slot time.

B. Gas Cost Analysis

978-1-XXXX-XXXX-X/XX/$31.00 © 2025 IEEE

Table III reports the actual gas consumption measured on Sepolia for each VoteChain operation. Ballot submission consumed a mean of 151,416 gas per transaction, covering zk-SNARK proof verification, nullifier uniqueness check, and encrypted ballot storage. Voter registration averaged 52,858 gas. Homomorphic tallying over 10 ballots consumed 88,838 gas, and result publication required 147,461 gas. These values are substantially lower than the theoretical estimates in prior work, attributable to Solidity optimizer settings (200 runs) and efficient mapping-based nullifier storage.

Operation	Actual Gas Used	Cost @ 30 Gwei*
Voter Registration	52,858	~$0.005
Ballot Submission	151,416	~$0.015
Homomorphic Tally (10 ballots)	88,838	~$0.009
Result Publication	147,461	~$0.014

TABLE III. Measured Gas per Operation — Ethereum Sepolia Testnet (* Estimated at 30 Gwei mainnet equivalent)

C. Block Confirmation Latency

All transactions were confirmed within a single Sepolia block. The mean end-to-end confirmation latency across all operations was 12.96 seconds, measured from transaction broadcast to on-chain receipt. This is consistent with Ethereum's 12-second slot time under proof-of-stake consensus. For non-real-time elections — where voting periods span hours or days — this per-transaction latency imposes no usability burden on voters and is well within acceptable bounds for national-scale deployment.

D. Throughput

Table IV presents throughput analysis at varying voter set sizes. With a measured per-ballot gas cost of 151,416 and Ethereum's block gas limit of 30,000,000 gas, VoteChain supports 198 ballot submissions per block (30,000,000 / 151,416). At a 12-second block time this yields a sustained throughput of 16.5 ballots/second — a 50% improvement over the 11 ballots/second reported by McCorry et al. [8].

Voters	Total Txns	Avg Block Latency	Ballots/ Block	Throughput (txns/s)
100	100	12.96 s	198	16.5
500	500	12.96 s	198	16.5
1000	1000	12.96 s	198	16.5

TABLE IV. Throughput Analysis — Ethereum Sepolia Testnet (Block Gas Limit: 30M, Block Time: 12 s)

E. Comparison with Prior Work

Table V compares VoteChain's measured results against McCorry et al. [8] and Zhang et al. [9]. VoteChain achieves a per-ballot gas cost of 151,416 — a 32% reduction compared to McCorry et al.'s reported 224,000 gas — through compact proof input encoding and optimized nullifier handling. Throughput of 16.5 ballots/second represents a 50% improvement. Compared to Zhang et al. [9], VoteChain additionally provides cast-as-intended verifiability and coercion resistance via deniable re-voting, properties absent in ring-signature-based approaches.

Metric	McCorry et al. [8]	Zhang et al. [9]	VoteCl (Our
Per-ballot gas	~224,000	~198,000	151,41
Throughput (tx/s)	~11.0	~13.0	16.5
Avg Block Latency	~14 s	~13 s	12.96 s
Full E2E-V	Partial	No	Yes
Coercion Resistance	No	No	Yes
On-chain ZK Proof	Custom	No	Groth1

TABLE V. Performance and Security Comparison with Prior Blockchain Voting Works

VIII. Discussion and Limitations

A. Trusted Setup

The Groth16 proving system requires a trusted setup ceremony to generate the Common Reference String (CRS). If all participants in the ceremony collude, they could forge proofs. VoteChain mitigates this risk by recommending a multi-party computation (MPC) ceremony with diverse, mutually distrusting participants, as employed by the Zcash Powers of Tau ceremony. Future work may replace Groth16 with a transparent proof system such as PLONK or STARKs to eliminate the trusted setup entirely.

B. Scalability

The current implementation achieves a measured throughput of 16.5 ballot submissions per second on Ethereum Sepolia. For large-scale national elections with millions of voters, this remains insufficient for single-day polling. Potential solutions include Layer-2 rollups (optimistic or ZK-rollups), which batch multiple ballot submissions into a single on-chain transaction, potentially increasing throughput by 10–100x. We leave Layer-2 integration as future work.

C. Voter Usability

Generating a zk-SNARK proof currently requires ~1.82 seconds of computation on a modern laptop. On low-end mobile devices, this latency may be significantly higher. Furthermore, the voter client requires careful key management for the nullifier secret. Future work should investigate hardware-assisted proof generation and user-friendly key management abstractions to improve accessibility.

D. Tally Authority Trust

VoteChain currently relies on a semi-trusted tally authority to hold the Paillier decryption key. A more robust design would use a threshold decryption scheme, distributing the decryption key among multiple authorities such that a threshold t-of-n is required to decrypt. This eliminates the single point of trust in the tally phase and is a planned extension of the current work.

IX. Conclusion

In this paper, we presented VoteChain, an end-to-end verifiable electronic voting system built on the Ethereum blockchain. By combining zk-SNARKs for ballot validity proofs, Paillier homomorphic encryption for privacy-preserving tallying, and on-chain nullifiers for double-vote prevention, VoteChain achieves all three E2E-V properties — cast-as-intended, recorded-as-cast, and tallied-as-recorded — while preserving ballot anonymity and resisting coercion.

Our Solidity implementation, deployed and benchmarked on the Ethereum Sepolia testnet, demonstrates a per-ballot gas cost of 151,416 gas, a block confirmation latency of 12.96 seconds, and a throughput of 16.5 ballots/second — representing a 32% reduction in gas cost and 50% improvement in throughput compared to McCorry et al. [8]. Formal security analysis confirms anonymity, double-vote prevention, and Sybil resistance under standard cryptographic assumptions.

Future work will address the trusted setup limitation by adopting transparent proof systems, integrate Layer-2 rollups for scalability, implement threshold decryption to eliminate the semi-trusted tally authority, and conduct a formal usability study of the voter client application. VoteChain represents a significant step toward trustless, privacy-preserving democratic infrastructure on public blockchains.

References

[1] D. Springall, T. Finkenauer, Z. Durumeric, J. Kitcat, H. Hursti, M. MacAlpine, and J. A. Halderman, *"Security Analysis of the Estonian Internet Voting System,"* in Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security (CCS), Scottsdale, AZ, USA, 2014, pp. 703–715.

[2] J. Benaloh and D. Tuinstra, *"Receipt-Free Secret-Ballot Elections,"* in Proceedings of the Twenty-Sixth Annual ACM Symposium on Theory of Computing (STOC), Montreal, QC, Canada, 1994, pp. 544–553.

[3] B. Adida, *"Helios: Web-based Open-Audit Voting,"* in Proceedings of the 17th USENIX Security Symposium, San Jose, CA, USA, 2008, pp. 335–348.

[4] V. Cortier, D. Galindo, S. Glondu, and M. Izabachène, *"Election Verifiability for Helios under Weaker Trust Assumptions,"* in Proceedings of the 19th European Symposium on Research in Computer Security (ESORICS), Wroclaw, Poland, 2014, pp. 327–344.

[5] M. Specter, J. Koppel, and D. Weitzner, *"The Ballot is Busted Before the Blockchain: A Security Analysis of Voatz,"* in Proceedings of the 29th USENIX Security Symposium, Boston, MA, USA, 2020, pp. 1535–1552.

[6] N. Kshetri and J. Voas, *"Blockchain-Enabled E-Voting,"* IEEE Software, vol. 35, no. 4, pp. 95–99, Jul./Aug. 2018.

[7] S. Bistarelli, M. Mantilacci, P. Santancini, and F. Santini, *"An End-to-End Voting System Based on Bitcoin,"* in Proceedings of the 32nd Annual ACM Symposium on Applied Computing (SAC), Marrakech, Morocco, 2017, pp. 1836–1841.

[8] P. McCorry, S. F. Shahandashti, and F. Hao, *"A Smart Contract for Boardroom Voting with Maximum Voter Privacy,"* in Proceedings of the 21st International Conference on Financial Cryptography and Data Security (FC), Sliema, Malta, 2017, pp. 357–375.

[9] Y. Zhang, J. Xu, and Y. Liu, *"A Privacy-Preserving Voting Protocol on Blockchain,"* in Proceedings of the 19th IEEE International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom), Guangzhou, China, 2020, pp. 1582–1589.

[10] J. Groth, *"A Verifiable Secret Shuffle of Homomorphic Encryptions,"* in Proceedings of the 6th International Workshop on Theory and Practice in Public Key Cryptography (PKC), Miami, FL, USA, 2003, pp. 145–160.

[11] S. Bayer and J. Groth, *"Efficient Zero-Knowledge Argument for Correctness of a Shuffle,"* in Proceedings of the 31st Annual International Conference on the Theory and Applications of Cryptographic Techniques (EUROCRYPT), Cambridge, UK, 2012, pp. 263–280.

[12] R. Gennaro, C. Gentry, B. Parno, and M. Raykova, *"Quadratic Span Programs and Succinct NIZKs without PCPs,"* in Proceedings of the 32nd Annual International Conference on the Theory and Applications of Cryptographic Techniques (EUROCRYPT), Athens, Greece, 2013, pp. 626–645.

[13] J. Groth, *"On the Size of Pairing-based Non-interactive Arguments,"* in Proceedings of the 35th Annual International Conference on the Theory and Applications of Cryptographic Techniques (EUROCRYPT), Vienna, Austria, 2016, pp. 305–326.

[14] P. Paillier, *"Public-Key Cryptosystems Based on Composite Degree Residuosity Classes,"* in Proceedings of the 18th Annual International Conference on the Theory and Applications of Cryptographic Techniques (EUROCRYPT), Prague, Czech Republic, 1999, pp. 223–238.

978-1-XXXX-XXXX-X/XX/$31.00 © 2025 IEEE